

Background

This notebook demonstrates how to use **X7Configuration**. The **X7Configuration** tool lets you input high level user parameters and convert it into a set of lower level X7 chip parameters that can be used to control the X7 module through the RadarDirect SDK.

The user parameters are divided into two configuration objects:

- X7UserConfiguration
- X7TxRxConfiguration

The **X7UserConfiguration** consists of parameters which control the radar frame parameters, e.g. FPS (frame rate), the range of the radar frame, and the frame integration duty cycle.

The **X7TxRxConfiguration** is used to configure which transmitters (Tx) and receivers (Rx) are active. The X7 has 2 Txs and 2 Rxs which can be configured independently. Only a single Tx can be active at a time, but Tx0 and Tx1 can be alternated in consecutive frames. Both Rxs can be active simultaneously.

Note that power management to optimize for low power consumption is not supported in the current release. Currently, the radar and on-board processor remains active between radar frames and processing. The power management feature will be added later.

X7UserConfiguration

The X7UserConfiguration has the following properties

- fps
- duty_cycle
- max_range
- antenna_gain
- pulse_period

fps

The **fps** parameter configures the number of radar frames output per second.

A higher **fps** gives the ability to resolve faster moving targets and respond more quickly to changes in the environment. However, higher **fps** usually also means higher power consumption and processing requirements, so there is a trade-off between performance and cost.

Note that the **fps** is the frame rate for each entry in the sequence, meaning that a sequence of length **N** will in practice give a data output rate of **N*fps** frames received per second. If the X7 is set up in a dual alternating Tx configuration with **10.0** FPS per Tx, the resulting output data rate will be $2 * 10.0 = 20.0$ FPS. However, the Nyquist rate per Tx channel corresponds to the configured **fps** and will in this example be **10.0** FPS.

In **X7Configuration** the **fps** parameter can be set in the interval **[1.0, 500.0]**, however due to the relationship between the X7 system clocks, the PRF (pulse repetition frequency) and the FPS, certain combinations of PRF and FPS will lead to spurious noise in the system. **X7Configuration** will therefore adjust the desired user **fps** to a valid setting based on the rest of the configuration. Please refer to the examples below to see how this is done. Note that the maximum **fps** is limited to the bandwidth of the SPI interface, and will vary depending on the frame length (**mframes_per_pulse**), and the host's ability to read out the frames.

duty_cycle

Describes the ratio of the time the radar is transmitting, compared to the total time between frames.

The duty cycle is expressed as a ratio, with a lower ratio indicating a longer off-time. The duty cycle will have an impact on the power consumption and overall performance of the radar system. Higher duty cycle usually gives higher signal-to-noise ratio (SNR) but longer active time of the transceivers giving higher power consumption.

The duty cycle is limited to values in the interval `[0.001, 1.0]`, however note that due to overhead related to processing and readout, a duty cycle of `1.0` is not practically possible and the configuration needs to be tested on the actual radar. Note that when a dual Tx configuration is used, radar frame interleaving adds an additional level of overhead which limits the maximum duty cycle (see the **number_of_interleaved_frames** section below).

min_range

The start range of the radar frame in meters, also referred to as range offset.

This parameter is currently not configurable. It is set to `-1.38` meters, as a result of the configuration of the Tx and Rx relative delays, and intrinsic delays in the X7 chip and the X7F202 module.

max_range

The maximum range of the radar frame in meters.

This parameter controls the maximum range of the radar to the lowest possible value that includes the **max_range** specified by the user. The input value is rounded up to the nearest multiple of `1.143` meters, which is the discrete step size of the valid values. Unwanted rangebins can be manually discarded by the user. The **max_range** can be set in the interval `[0.1, 12.33]` meters.

antenna_gain

The one-way antenna gain for the radar module in linear scale.

This parameter is used to calculate the **tx_power** level in the transmitter recommended for regulatory compliance. Note that users of the X7 radar are responsible for ensuring regulatory compliance in the relevant regions when designing a product. For the X7F202 the peak **antenna_gain** is `2.595` in linear scale (at 0 degrees elevation, $\sim \pm 20$ degrees azimuth, corresponds to $10 \cdot \log_{10}(2.595) = 4.141$ dB). Note that the **antenna_gain** should be always be greater than 0.0 (linear scale).

pulse_period

Determines the pulse repetition frequency (PRF) of the transmitter(s).

The system has a `131.25` MHz TRX clock and the PRF is calculated as $PRF = 131.25 \text{ MHz} / \text{pulse_period}$. The PRF determines the unambiguous range **Rmax** of the radar frame, which can be calculated by $R_{max} = c / (2 \cdot PRF)$, where **c** is the speed of light in vacuum. If another pulse is transmitted before a target reflection from the previous one has had the chance to return to the receiver, the correct target range will not be possible to unambiguously resolve. The **pulse_period** must therefore result in an $R_{max} \geq \text{max_range}$, where **max_range** is the derived rounded value explained above. In the end, this means that $\text{pulse_period} \geq \text{mframes_per_pulse}$, where **mframes_per_pulse** is derived from the desired **max_range** and is explained in the **X7ChipConfiguration** section below.

The **pulse_period** parameter can be calculated automatically by setting it to `0` in the user configuration, or not supplying it to the **X7UserConfiguration** constructor. Please see the examples below. The **pulse_period** parameter can be set in the interval `[4, 16]`.

X7TxRxConfiguration

The X7TxRxConfiguration has the following properties

- `number_of_chips`
- `tx_channel_sequence`
- `rx_mask_sequence`

number_of_chips

The number of X7 chips in the setup.

Multiple X7 chips can be connected and synchronized on the same HW module. In order to control the Tx/Rx sequencing, the **number_of_chips** in the setup needs to be known. On the X7F202 module, there's only 1 X7 chip.

The **number_of_chips** parameter can be set to **1** or **2**.

tx_channel_sequence

The sequence of active Tx channels.

The X7 only supports a single Tx channel being active at a time. This parameter controls the sequence of active Txs as an array of integers. Valid Tx channel configurations are:

Description	Integer value	Hexadecimal value
Both Txs off	65535	0xFFFF
Tx0 active	0	0x0
Tx1 active	1	0x1

rx_mask_sequence

The sequence of Rx masks.

This parameter allows you to control the active Rxs for each entry in the sequence. The mask is a bitmask represented as an integer. Valid Rx mask configurations are:

Description	Integer value	Bitmask
Both Rxs off	0	0b00
Rx0 active only	1	0b01
Rx1 active only	2	0b10
Both Rxs active	3	0b11

Examples of Tx/Rx sequence configurations

Working configurations

There are currently issues with certain combinations of Tx/Rx sequences. The table lists most of the working tested configurations. Depending on the application, other sequences than the ones listed here can also be created, based on working configurations. The maximum sequence length is currently set to **4**.

Description	TxChannelSequence	RxMaskSequence
Both Txs off, single Rx0	[65535]	[1]
Both Txs off, dual Rx0/Rx1	[65535]	[3]
Single Tx0 , single Rx0	[0]	[1]
Single Tx1 , single Rx0	[1]	[1]
Single Tx0 , dual Rx0/Rx1	[0]	[3]
Single Tx1 , dual Rx0/Rx1	[1]	[3]
Dual alternating Tx0/Tx1 , single Rx0	[0, 1]	[1, 1]
Dual alternating Tx0/Tx1 , dual Rx0/Rx1	[0, 1]	[3, 3]

Known non-working configurations

This is a list of some of the known non-working configurations. Sequences including these configurations will not work.

Description	TxChannelSequence	RxMaskSequence
Both Txs off, single Rx1	[65535]	[2]
Single Tx0 , single Rx1	[0]	[2]
Single Tx1 , single Rx1	[1]	[2]

X7ChipConfiguration

This section briefly describes some of the lower level X7 chip configuration parameters part of the **X7ChipConfiguration**. In addition to fields already described in the sections above, the **X7ChipConfiguration** consists of the following properties

- iterations_per_frame
- pulses_per_iteration
- mframes_per_pulse
- number_of_interleaved_frames
- tx_power

iterations_per_frame and pulses_per_iteration

For every radar frame, a number of radar pulses is transmitted with the pulse repetition frequency (PRF). The number of pulses transmitted per frame is controlled by the two parameters **iterations_per_frame** and **pulses_per_iteration**.

The radar pulses are sent in iterations, controlled by **iterations_per_frame**, and each of the iterations consists of **pulses_per_iteration** pulses. The total number of pulses is given by `iterations_per_frame * pulses_per_iteration ( * number_of_interleaved_frames)`. **number_of_interleaved_frames** is only used if a dual Tx configuration is used (see the explanation below). Together with the PRF and the FPS, these parameters are calculated based on the desired **duty_cycle** specified in **X7UserConfiguration**.

mframes_per_pulse

mframes_per_pulse controls the number of microframes in the radar frame. Each microframe consists of 16 range bins, so this parameter controls the number of total range bins in the radar frame and is calculated based on the desired **max_range** specified in **X7UserConfiguration**.

number_of_interleaved_frames

The X7 doesn't support phase shifts on the Tx side, but virtual beamforming can be accomplished by summing the frames from the 2 Txs after Rx beamforming. However, if the time between **Tx0** and **Tx1** is too high and the target is moving at for instance walking speed, there will be a phase difference between **Tx0** and **Tx1** caused by the moving target. The faster the target moves the bigger the difference phase.

To reduce this effect the FPS can be increased, reducing the time between **Tx0** and **Tx1**, however this requires processing data at a higher rate. There is also a limitation to the bandwidth we can sustain over the SPI interface between X7F202 and the host, limiting the FPS. To mitigate this problem, the concept of frame interleaving is introduced. Interleaving divides the **Tx0/Tx1** sequencing into subframes. Instead of first sending all **Tx0** pulses, then all **Tx1** pulses, the transmitters switch at a higher interval back and forth to average out the difference phase effect. How many subframes are interleaved per Tx is controlled by **number_of_interleaved_frames**.

Note that in the case of single Tx the interleaving concept doesn't make any sense, in which case **number_of_interleaved_frames** is set to `0`. Due to how the sampling system works, together with the clock system, **number_of_interleaved_frames** is automatically set to `5` in the case of running a dual Tx sequence, although setting `10` is also valid.

Also note that doing interleaving adds some processing overhead between each radar frame, effectively reducing the upper limit to the **duty_cycle**, meaning that the resulting configuration needs to be tested on an actual radar to ensure that it's valid.

tx_power

In the X7 transmitters, the **tx_power** can be set to 4 different levels, 1 to 4, where 1 gives the lowest power level transmitted, and 4 gives the highest. Given the PRF, **iterations_per_frame**, **pulses_per_iteration** and potentially **number_of_interleaved_frames**, the maximum **tx_power** is calculated to give a transmitted power spectral density within the maximum levels controlled by UWB regulations.

Note that, as explained above, there is no guarantee that the settings provided here will be within the regulations. These vary for different regions and Novelda can only provide recommendations which need to be qualified in proper regulatory compliance tests.

Examples

In this section, some example configurations of the X7 radar is shown using **X7Configuration** with the **pyx7configuration** binding. First, let's set up some of the base variables valid for all examples.

```
In [1]: from pyx7configuration import X7Configuration
        from pyx7configuration import X7UserConfiguration
        from pyx7configuration import X7TxRxConfiguration

        antenna_gain_x7f202 = 2.595 # module dependent antenna gain parameter. Other modules might have other values
        number_of_chips = 1 # X7F202 module has 1 X7 chip
```

1D mode 4 FPS

In this example the radar is set up in single channel **Tx0/Rx0** mode with 4.0 FPS and 2.0 meter range. A low duty cycle is set to allow for lower power consumption.

First, let's set up the radar frame parameters by defining the **X7UserConfiguration** parameters.

```
In [2]: fps_1D = 4.0
        duty_cycle_1D = 0.001
        max_range_1D = 2.0
        userCfg_1D = X7UserConfiguration(
            fps_1D, duty_cycle_1D, max_range_1D, antenna_gain_x7f202
        )
        print(userCfg_1D)
```

```
X7UserConfiguration
      fps = 4.00
      duty_cycle = 0.001
      max_range = 2.00
      antenna_gain = 2.60
      pulse_period = 0
```

Next, let's define the Tx/Rx scheme by defining the **X7TxRxConfiguration**. By just using a single transceiver, **Tx0** and **Rx0**, only the range of the target, and not the angular position can be resolved, hence calling this a 1D setup.

```
In [3]: tx0_1D = [0]
        rx0_1D = [1]
        txrxCfg_1D = X7TxRxConfiguration(tx0_1D, rx0_1D, number_of_chips)
        print(txrxCfg_1D)
```

```
X7TxRxConfiguration
      number_of_active_tx_channels = 1
      number_of_active_rx_channels = 1
      number_of_chips = 1
      is_all_tx_off = False
      tx_channel_sequence = [0]
      rx_mask_sequence = [1]
```

This is all that's needed to find a set of chip parameters to input in the SDK. Lets find the chip parameters,

X7ChipConfiguration

```
In [4]: x7config_1D = X7Configuration(userCfg_1D, txrxCfg_1D)
chipCfg_1D = x7config_1D.get_chip_config()
print(chipCfg_1D)
```

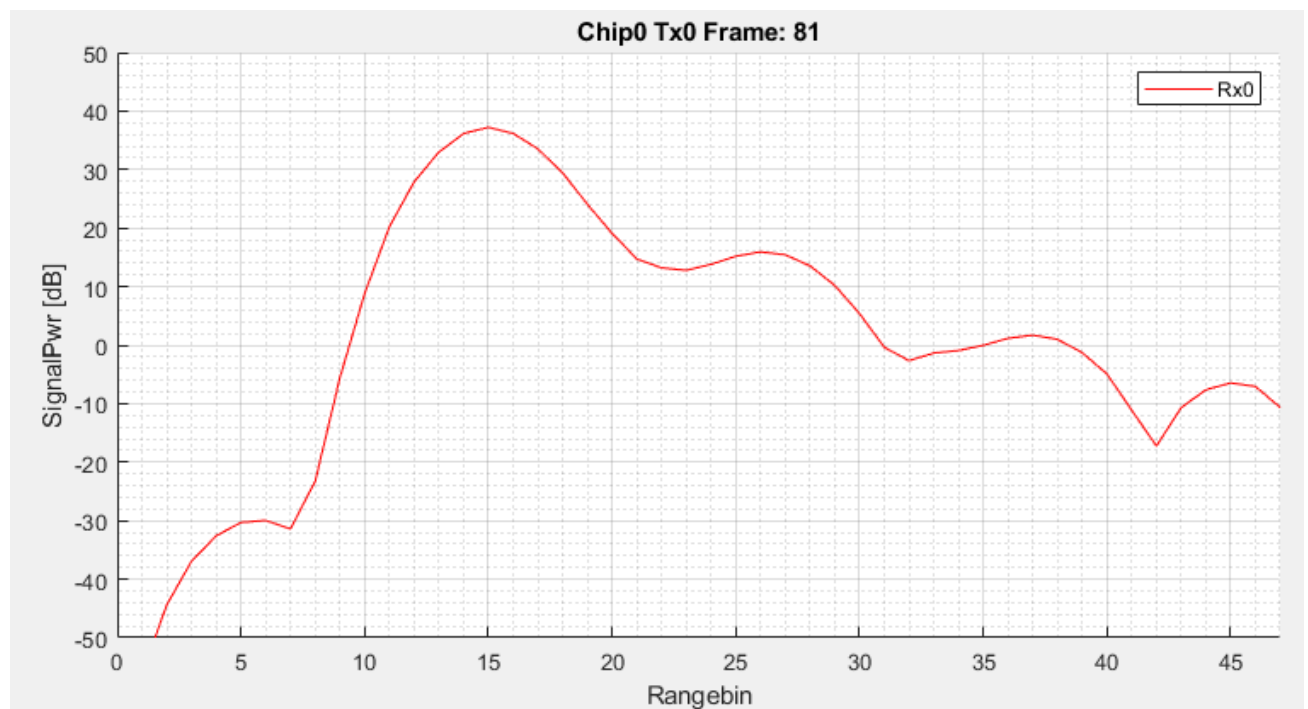
```
X7ChipConfiguration
  fps = 4.00
  iterations_per_frame = 234
  pulses_per_iteration = 35
  mframes_per_pulse = 3
  pulse_period = 4
  number_of_interleaved_frames = 0
  tx_power = 3
  tx_channel_sequence = [0]
  rx_mask_sequence = [1]
```

The **X7Configuration** will try to find the closest matching parameters under the constraints set by the X7 chip and the limitations of the FW in the current release. The resulting **X7UserConfiguration** based on the calculated **X7ChipConfiguration** can be found using:

```
In [5]: userCfg_1D_resulting = x7config_1D.get_user_cfg()
print(userCfg_1D_resulting)
```

```
X7UserConfiguration
  fps = 4.00
  duty_cycle = 0.001
  max_range = 2.05
  antenna_gain = 2.60
  pulse_period = 0
```

The resulting **duty_cycle** is close to the input parameter but the resulting **max_range** is always greater than the desired **max_range**. Baseband data should now stream from the radar module. Below, the amplitude of a baseband radar frame with the above settings is visualized in dB.



2D mode 10 FPS

In this example, the radar is set up with both transceivers active, allowing target detection in 2D using beamforming or angle-of-arrival methods.

This is the high-level setup

- 2 Tx and 2 Rx.
- Range is set close to the maximum (12 meters).
- A relatively low duty cycle to maintain some low power performance.

For **X7UserConfiguration** this gives:

```
In [6]: fps_2D = 10.0
duty_cycle_2D = 0.03
max_range_2D = 12.0
userCfg_2D = X7UserConfiguration(
    fps_2D, duty_cycle_2D, max_range_2D, antenna_gain_x7f202
)
print(userCfg_2D)
```

```
X7UserConfiguration
  fps = 10.00
  duty_cycle = 0.030
  max_range = 12.00
  antenna_gain = 2.60
  pulse_period = 0
```

Next, the **X7TxRxConfiguration**:

```
In [7]: tx_2D = [0, 1] # tx0, tx1 sequencing
rx_2D = [3, 3] # both Rx active simultaneously --> bitmask = 0b11
txrxCfg_2D = X7TxRxConfiguration(tx_2D, rx_2D, number_of_chips)
print(txrxCfg_2D)
```

```
X7TxRxConfiguration
  number_of_active_tx_channels = 2
  number_of_active_rx_channels = 2
  number_of_chips = 1
  is_all_tx_off = False
  tx_channel_sequence = [0,1]
  rx_mask_sequence = [3,3]
```

This gives the following **X7ChipConfiguration**:

```
In [8]: x7config_2D = X7Configuration(userCfg_2D, txrxCfg_2D)
chipCfg_2D = x7config_2D.get_chip_config()
print(chipCfg_2D)
```

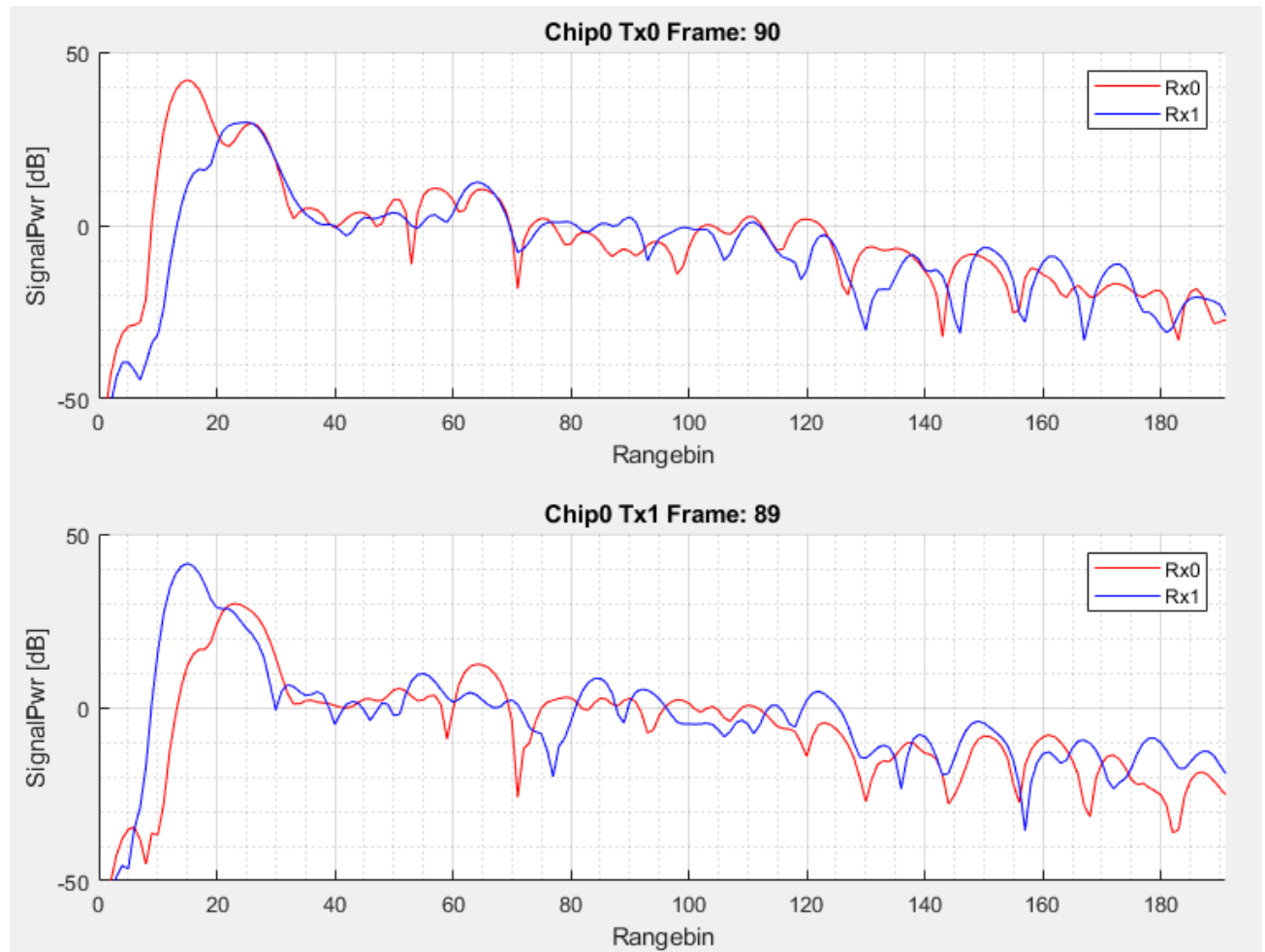
```
X7ChipConfiguration
  fps = 10.00
  iterations_per_frame = 92
  pulses_per_iteration = 35
  mframes_per_pulse = 12
  pulse_period = 12
  number_of_interleaved_frames = 5
  tx_power = 3
  tx_channel_sequence = [0,1]
  rx_mask_sequence = [3,3]
```

and the following resulting **X7UserConfiguration**:

```
In [9]: userCfg_2D_resulting = x7config_2D.get_user_config()
print(userCfg_2D_resulting)
```

```
X7UserConfiguration
  fps = 10.00
  duty_cycle = 0.029
  max_range = 12.33
  antenna_gain = 2.60
  pulse_period = 0
```

Below, the amplitude of the baseband radar frames from the two Rxs for both of the Txs are visualized in dB.



The examples above have shown how the resulting user configuration adjusts the parameters to satisfy what the user wants, while still ensuring that the complex interdependent relationship between the low level chip parameters is correct.

Because of how the clock system is set up in the X7 radar chip, spurious noise can occur depending on the relationship between the FPS and PRF and the system clocks. To see this in practice, let's try to adjust the **fps** to something else.

```
In [10]: userCfg_2D.fps = 9.0
         print(userCfg_2D)
```

```
X7UserConfiguration
  fps = 9.00
  duty_cycle = 0.030
  max_range = 12.00
  antenna_gain = 2.60
  pulse_period = 0
```

Now, let's see what the **X7ChipConfiguration** looks like.

```
In [11]: x7config_2D = X7Configuration(userCfg_2D, txrxCfg_2D)
         chipCfg_2D = x7config_2D.get_chip_config()
         print(chipCfg_2D)
```

```
X7ChipConfiguration
    fps = 10.00
    iterations_per_frame = 92
    pulses_per_iteration = 35
    mframes_per_pulse = 12
    pulse_period = 12
    number_of_interleaved_frames = 5
    tx_power = 3
    tx_channel_sequence = [0,1]
    rx_mask_sequence = [3,3]
```

In this specific example, the **fps** is adjusted from the input user parameter `9.0` to instead be `10.0`, to ensure no spurious noise based on the explanation above.

To see the available **fps** values for a given **pulse_period**, `find_valid_fps_list_from_pulse_period()` can be used

```
In [12]: fps_list = x7config_2D.find_valid_fps_list_from_pulse_period(chipCfg_2D.pulse_period)
print(fps_list)
```

```
[1.0, 2.0, 4.0, 5.0, 10.0, 20.0, 25.0, 50.0, 100.0, 125.0, 250.0, 483.0, 486.0, 500.0]
```

Let's now see how the FPS list is affected by setting different **pulse_period** values.

```
In [13]: for pulse_period in range(4, 16+1):
    fps_list = x7config_2D.find_valid_fps_list_from_pulse_period(pulse_period)
    print(
        "pulse_period = " + str(pulse_period) + "\nfps_list = " + str(fps_list) + "\n"
    )
```

```

pulse_period = 4
fps_list = [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 10.0, 12.0, 15.0, 20.0, 25.0, 30.0, 50.0, 60.0, 75.0, 100.0, 125.0, 150.0, 207.0, 250.0, 300.0, 375.0, 483.0, 485.0, 486.0, 500.0]

pulse_period = 5
fps_list = [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 8.0, 10.0, 12.0, 15.0, 16.0, 20.0, 24.0, 25.0, 30.0, 40.0, 48.0, 50.0, 60.0, 68.0, 75.0, 80.0, 100.0, 120.0, 125.0, 131.0, 150.0, 152.0, 193.0, 200.0, 207.0, 240.0, 248.0, 250.0, 252.0, 262.0, 265.0, 267.0, 276.0, 290.0, 300.0, 307.0, 322.0, 335.0, 339.0, 343.0, 349.0, 365.0, 367.0, 375.0, 377.0, 386.0, 388.0, 393.0, 400.0, 412.0, 414.0, 443.0, 444.0, 445.0, 467.0, 483.0, 485.0, 486.0, 490.0, 492.0, 499.0, 500.0]

pulse_period = 6
fps_list = [1.0, 2.0, 4.0, 5.0, 8.0, 10.0, 20.0, 25.0, 40.0, 50.0, 100.0, 125.0, 131.0, 200.0, 250.0, 322.0, 367.0, 412.0, 483.0, 486.0, 500.0]

pulse_period = 7
fps_list = [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 8.0, 10.0, 12.0, 15.0, 16.0, 20.0, 24.0, 25.0, 30.0, 40.0, 48.0, 50.0, 60.0, 68.0, 75.0, 80.0, 100.0, 120.0, 125.0, 131.0, 150.0, 152.0, 193.0, 200.0, 207.0, 240.0, 248.0, 250.0, 252.0, 262.0, 265.0, 267.0, 276.0, 290.0, 300.0, 307.0, 322.0, 335.0, 339.0, 343.0, 349.0, 365.0, 367.0, 375.0, 377.0, 386.0, 388.0, 393.0, 400.0, 412.0, 414.0, 443.0, 444.0, 445.0, 467.0, 483.0, 485.0, 486.0, 490.0, 492.0, 499.0, 500.0]

pulse_period = 8
fps_list = [1.0, 2.0, 3.0, 5.0, 6.0, 10.0, 15.0, 25.0, 30.0, 50.0, 75.0, 125.0, 150.0, 250.0, 375.0]

pulse_period = 9
fps_list = [322.0, 499.0]

pulse_period = 10
fps_list = [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 8.0, 10.0, 12.0, 15.0, 20.0, 24.0, 25.0, 30.0, 40.0, 50.0, 60.0, 75.0, 100.0, 120.0, 125.0, 131.0, 150.0, 193.0, 200.0, 207.0, 250.0, 300.0, 322.0, 335.0, 339.0, 365.0, 367.0, 375.0, 393.0, 412.0, 414.0, 444.0, 467.0, 483.0, 485.0, 486.0, 492.0, 500.0]

pulse_period = 11
fps_list = [365.0, 467.0]

pulse_period = 12
fps_list = [1.0, 2.0, 4.0, 5.0, 10.0, 20.0, 25.0, 50.0, 100.0, 125.0, 250.0, 483.0, 486.0, 500.0]

pulse_period = 13
fps_list = [131.0, 262.0, 343.0, 367.0, 393.0]

pulse_period = 14
fps_list = [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 8.0, 10.0, 12.0, 15.0, 20.0, 24.0, 25.0, 30.0, 40.0, 50.0, 60.0, 75.0, 100.0, 120.0, 125.0, 131.0, 150.0, 193.0, 200.0, 207.0, 250.0, 300.0, 322.0, 335.0, 339.0, 365.0, 367.0, 375.0, 393.0, 412.0, 414.0, 444.0, 467.0, 483.0, 485.0, 486.0, 492.0, 500.0]

pulse_period = 15
fps_list = [1.0, 2.0, 4.0, 5.0, 8.0, 10.0, 16.0, 20.0, 25.0, 40.0, 50.0, 80.0, 100.0, 125.0, 131.0, 200.0, 250.0, 262.0, 265.0, 276.0, 322.0, 367.0, 400.0, 412.0, 445.0, 483.0, 486.0, 490.0, 499.0, 500.0]

pulse_period = 16
fps_list = [1.0, 3.0, 5.0, 15.0, 25.0, 75.0, 125.0, 375.0]

```

AllTxOff mode

In this example the radar is configured with all Tx's off. This mode is used in order to estimate the receiver noise level for a given radar frame setup for use in detection algorithms. Let's re-use the same **X7UserConfiguration** settings as the example above.

```

In [14]: fps_2D = 10.0
duty_cycle_2D = 0.03
max_range_2D = 12.0
userCfg_2D = X7UserConfiguration(
    fps_2D, duty_cycle_2D, max_range_2D, antenna_gain_x7f202
)
print(userCfg_2D)

```

```

X7UserConfiguration
    fps = 10.00
    duty_cycle = 0.030
    max_range = 12.00
    antenna_gain = 2.60
    pulse_period = 0

```

Next, let's define the **X7TxRxConfiguration** with both Tx's turned off, and both Rx's active. This can be done in two ways; either by setting it to `0xFFFF = 65535` or by giving an empty `TxSequence` array. In the latter case **X7TxRxConfiguration** will insert `65535` in the sequence automatically.

```

In [15]: all_txs_off = [65535] # Alternatively; all_txs_off = [0xFFFF], or all_txs_off = []
         rx_2D = [3]
         txrxCfg_AllTxOff = X7TxRxConfiguration(all_txs_off, rx_2D, number_of_chips)
         print(txrxCfg_AllTxOff)

```

```

X7TxRxConfiguration
    number_of_active_tx_channels = 0
    number_of_active_rx_channels = 2
    number_of_chips = 1
    is_all_tx_off = True
    tx_channel_sequence = [65535]
    rx_mask_sequence = [3]

```

This gives the following **X7ChipConfiguration**:

```

In [16]: x7config_AllTxOff = X7Configuration(userCfg_2D, txrxCfg_AllTxOff)
         chipCfg_AllTxOff = x7config_AllTxOff.get_chip_config()
         print(chipCfg_AllTxOff)

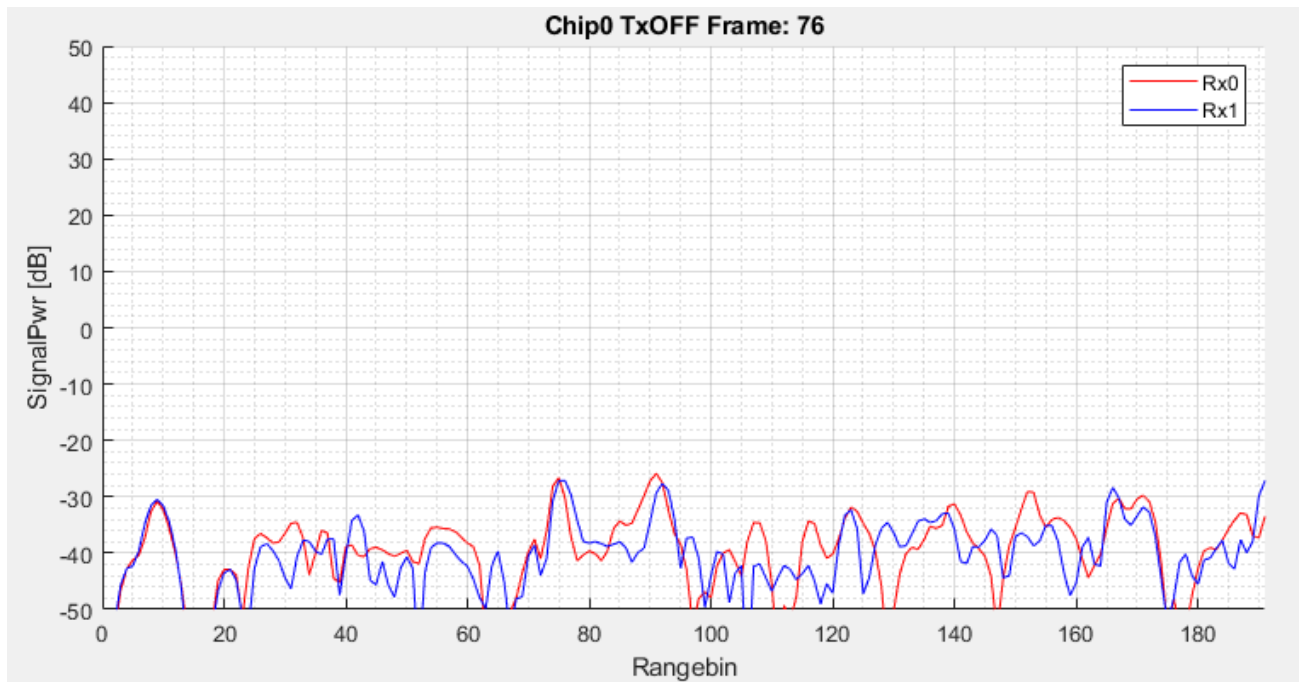
```

```

X7ChipConfiguration
    fps = 10.00
    iterations_per_frame = 936
    pulses_per_iteration = 35
    mframes_per_pulse = 12
    pulse_period = 12
    number_of_interleaved_frames = 0
    tx_power = 3
    tx_channel_sequence = [65535]
    rx_mask_sequence = [3]

```

Below, the amplitude of the baseband radar frames from the two Rx's with both Tx's disabled are visualized in dB. It's clearly visible that there are no reflections in the frame, since no radar pulses are actually sent in this mode.



2D mode with high FPS

In this example the radar is set up with a high FPS scheme intended for resolving walking targets using Doppler processing, e.g. Range-Doppler. When running both Tx's, interleaving will be automatically be enabled based on the explanation in the **X7ChipConfiguration** section.

This is the high-level setup

- 2 Tx's and 2 Rx's.
- Range is set to approximately 6.0 meters.
- A relatively high duty cycle to get high sensitivity.

For **X7UserConfiguration** this gives:

```
In [17]: fps_2D_high_fps = 250.0
duty_cycle_2D_high_fps = 0.7
max_range_2D_high_fps = 6.0
userCfg_2D_high_fps = X7UserConfiguration(
    fps_2D_high_fps, duty_cycle_2D_high_fps, max_range_2D_high_fps, antenna_gain_x7f202
)
print(userCfg_2D_high_fps)
```

```
X7UserConfiguration
  fps = 250.00
  duty_cycle = 0.700
  max_range = 6.00
  antenna_gain = 2.60
  pulse_period = 0
```

Next, the **X7TxRxConfiguration**:

```
In [18]: tx_2D_high_fps = [0, 1] # tx0, tx1 sequencing
rx_2D_high_fps = [3, 3] # both Rx active simultaneously --> bitmask = 0b11
txrxCfg_2D_high_fps = X7TxRxConfiguration(
    tx_2D_high_fps, rx_2D_high_fps, number_of_chips
)
print(txrxCfg_2D_high_fps)
```

```
X7TxRxConfiguration
    number_of_active_tx_channels = 2
    number_of_active_rx_channels = 2
    number_of_chips = 1
    is_all_tx_off = False
    tx_channel_sequence = [0,1]
    rx_mask_sequence = [3,3]
```

This gives the following **X7ChipConfiguration**:

```
In [19]: x7config_2D_high_fps = X7Configuration(userCfg_2D_high_fps, txrxCfg_2D_high_fps)
chipCfg_2D_high_fps = x7config_2D_high_fps.get_chip_config()
print(chipCfg_2D_high_fps)
```

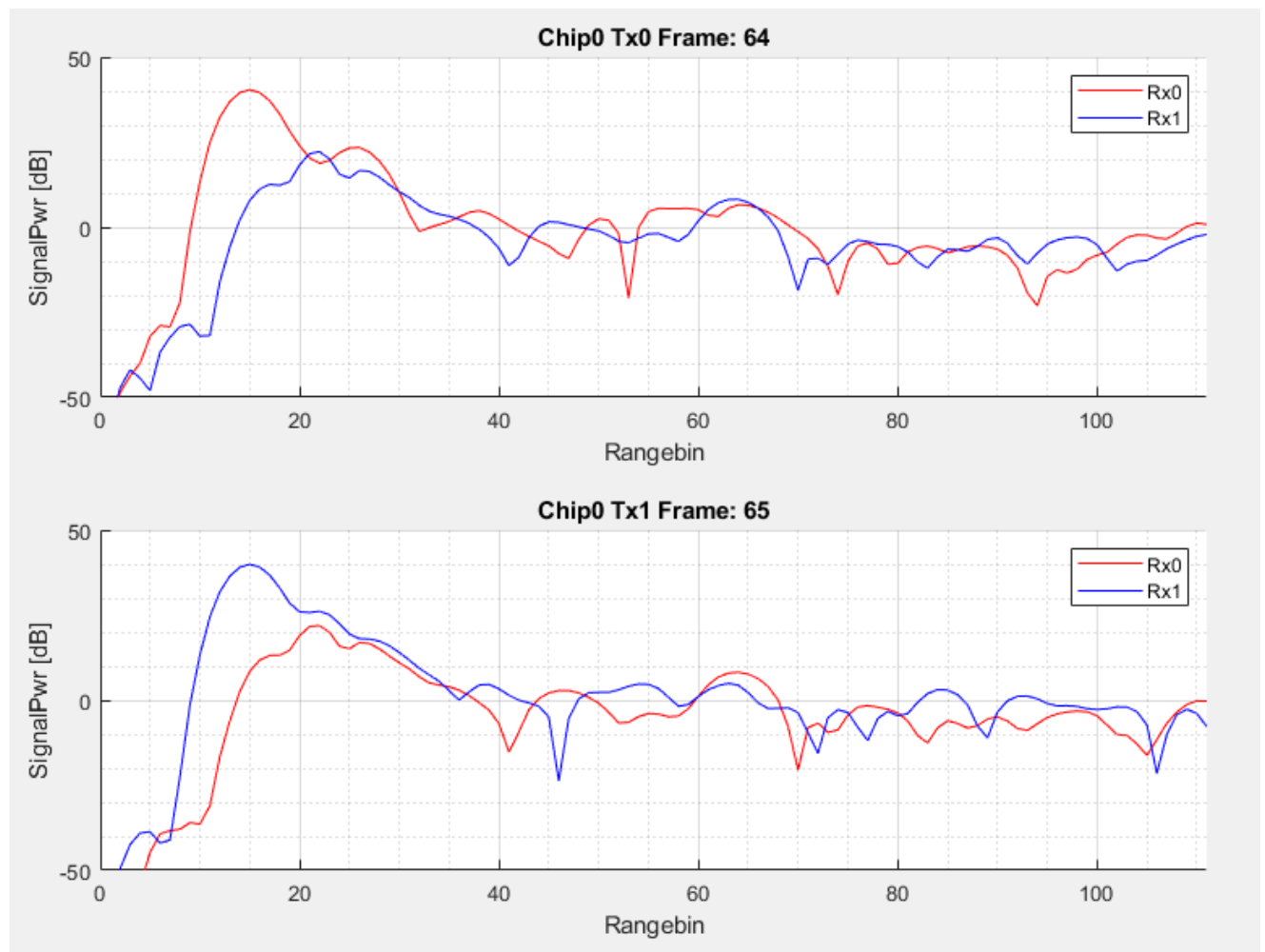
```
X7ChipConfiguration
    fps = 250.00
    iterations_per_frame = 148
    pulses_per_iteration = 35
    mframes_per_pulse = 7
    pulse_period = 7
    number_of_interleaved_frames = 5
    tx_power = 2
    tx_channel_sequence = [0,1]
    rx_mask_sequence = [3,3]
```

This is the resulting **X7UserConfiguration**:

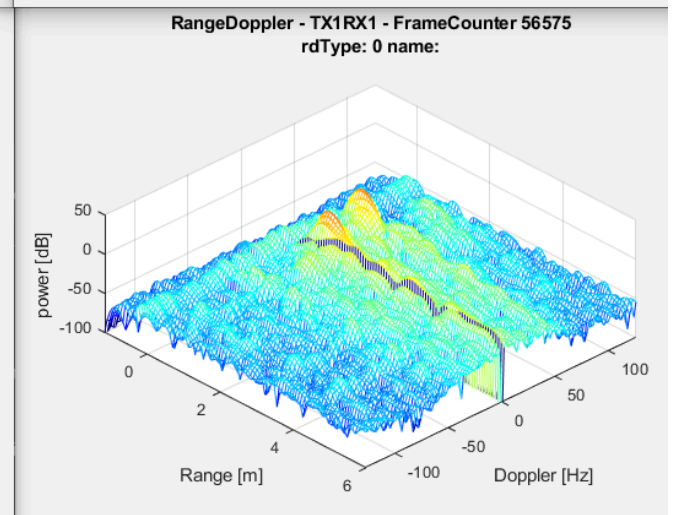
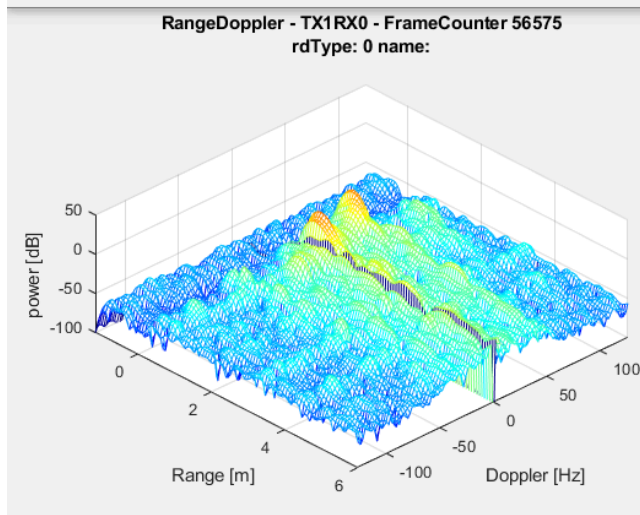
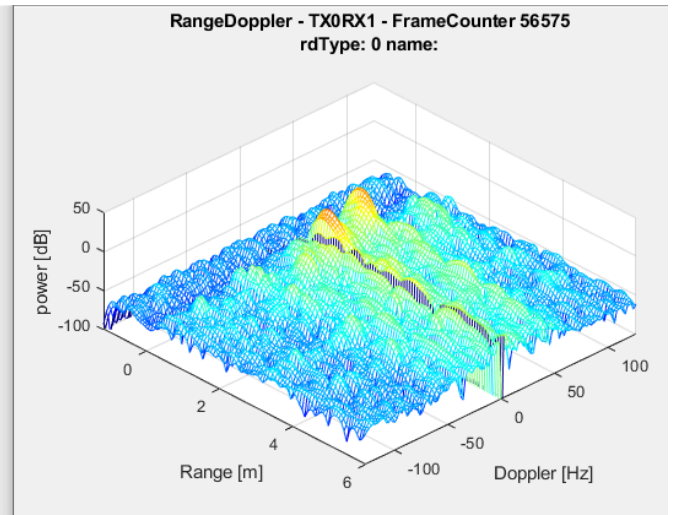
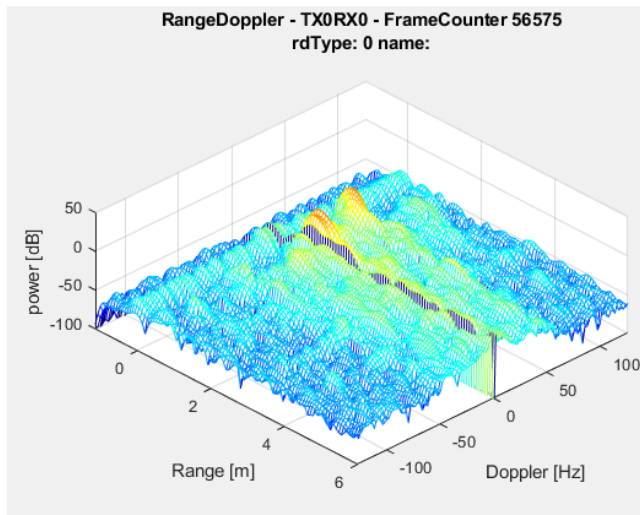
```
In [20]: userCfg_2D_high_fps_resulting = x7config_2D_high_fps.get_user_config()
print(userCfg_2D_high_fps_resulting)
```

```
X7UserConfiguration
    fps = 250.00
    duty_cycle = 0.691
    max_range = 6.62
    antenna_gain = 2.60
    pulse_period = 0
```

Below, the amplitude of the baseband radar frames from the two Rx's for both of the Tx's are visualized in dB.



Below, RangeDoppler matrices for all the four raw channels have been visualized.



The RangeDopplers have been calculated with the following parameters

- 128 frames buffered (~0.5 seconds @ 250 FPS)
- Hanning windowing
- Static removal with weighted mean
- 128 point FFTs